

Uchko: An Adaptive Mathematics Learning Platform with Predictive Modeling and Knowledge Tracing

Tarik Agić and Azra Kurić
Faculty of Electrical Engineering, University of Sarajevo
Sarajevo, Bosnia and Herzegovina
{tagic2, akuric1}@etf.unsa.ba

Abstract—Adaptive learning systems must connect predictive models with persistent learner state, curriculum-aligned content, and a usable delivery interface. This paper presents Uchko, a web platform for adaptive middle-school mathematics practice. The deployed hybrid combines an XGBoost classifier, which estimates the probability of an independently successful response, with Bayesian Knowledge Tracing (BKT), which maintains an interpretable mastery probability for each skill. A deterministic policy favors weak skills and problems whose predicted success is near a 0.65 target. All historical features use only prior interactions, and train, validation, and test partitions are disjoint at the student level. After filtering and metadata joins, the modeling data contain 1,693,947 interactions from 5,000 students. On 253,718 matched next-response test cases, XGBoost achieves 0.8125 ROC AUC, 0.7443 accuracy, and 0.5058 log loss. A compact gated recurrent Deep Knowledge Tracing (DKT) model reaches 0.7620 ROC AUC and 0.5542 log loss, outperforming a history baseline but not XGBoost. This controlled negative result supports retaining the more accurate and operationally simpler boosted-tree predictor. The deployed XGBoost–BKT hybrid is integrated into a Django application with role-based student and professor views, persistent attempts and sessions, curriculum exploration, sanitized imported content, and explicit separation between answer correctness and unaided success. Uchko demonstrates how comparative educational data mining experiments can inform an end-to-end adaptive learning implementation.

Index Terms—adaptive learning, educational data mining, knowledge tracing, Deep Knowledge Tracing, XGBoost, student modeling

I. INTRODUCTION

Students working through the same curriculum differ in prior knowledge, pace, and need for support. Adaptive learning attempts to address this variation by estimating learner state and selecting an appropriate next activity. The technical challenge is broader than predicting whether an answer will be correct: a useful platform must preserve student histories, represent curricular skills, provide actionable information to teachers, and convert model outputs into stable pedagogical decisions. Recent reviews therefore describe adaptive learning as a system-level problem spanning student modeling, content, sequencing, feedback, and governance [1], [2].

Knowledge tracing is central to this problem. Classical BKT models skill mastery as a latent variable updated after each observation [3]. Deep Knowledge Tracing (DKT) later demonstrated that recurrent networks can learn richer repre-

sentations directly from interaction sequences [4]. However, greater predictive capacity does not by itself determine how a system should choose content, communicate progress, or handle assisted answers. Interpretable state estimates and reliable software behavior remain particularly important in educational deployments.

This paper presents Uchko, an end-to-end adaptive mathematics platform created as an implementation and experimental demonstration. It builds on FoundationalASSIST, a recent dataset containing complete question content, student responses, curriculum alignment, and approximately 1.7 million interactions from 5,000 learners [5]. Uchko uses the protected interaction log only for offline model training; the application imports problem and skill metadata but does not distribute the interaction records.

The principal contributions are:

- a leakage-aware student-success model evaluated on entirely unseen learners;
- a controlled comparison between aggregate history, recurrent DKT, BKT, and gradient-boosted prediction under a common student-disjoint split;
- a hybrid design in which XGBoost ranks candidate difficulty while skill-specific BKT provides an interpretable, continuously updated mastery state;
- a concrete adaptive policy that targets productive challenge and avoids immediately repeating recently shown items;
- a persistent, role-aware Django implementation that exposes learning activities to students and aggregate progress to their professor; and
- a deliberate distinction between literal answer correctness and independent success, preventing hints or revealed answers from being counted as unaided evidence of mastery.

The goal is not to claim improved learning outcomes from an offline experiment. Rather, the work evaluates next-response prediction and demonstrates how the resulting models can be integrated responsibly into a usable adaptive workflow.

A. Related Work

Research on adaptive educational systems spans intelligent tutoring, learning analytics, and educational data mining. Sys-

tematic reviews find substantial technical activity but fewer evaluations that connect model improvements to learners’ real challenges and classroom deployment [1], [2]. ASSISTments is a particularly relevant foundation because it combines online mathematics practice with immediate feedback and teacher-facing information; a randomized evaluation reported improved student achievement from online mathematics homework using the platform [6]. Uchko does not attempt to reproduce that causal claim. It uses a newer ASSISTments-derived dataset to evaluate prediction and to implement a deployable adaptation loop.

Within knowledge tracing, BKT remains influential because its parameters have explicit cognitive interpretations, while DKT and later attentive models capture richer dependencies in student sequences [3], [4], [7]. Surveys emphasize that comparisons are sensitive to preprocessing, splitting, and model configuration [8]. Uchko therefore prioritizes a reproducible, student-disjoint evaluation and includes a controlled DKT experiment rather than assuming that a deep sequence model must be superior. The distinctive focus is not a novel knowledge-tracing architecture, but evidence-based selection and integration of complementary components into a tested application whose instructional policy is inspectable.

II. BACKGROUND AND DESIGN RATIONALE

A. Complementary Student Models

BKT represents each skill through four parameters: initial mastery $P(L_0)$, learning $P(T)$, guessing $P(G)$, and slipping $P(S)$. Given the current mastery $P(L)$, the probability of a correct response is

$$P(C) = P(L)(1 - P(S)) + (1 - P(L))P(G). \quad (1)$$

After observing a response, Bayes’ rule updates mastery, followed by a learning transition. BKT is compact and pedagogically legible, but its conditional independence assumptions restrict the interactions it can represent.

Gradient-boosted trees provide a complementary capability. XGBoost constructs an additive ensemble of regularized decision trees and is designed for scalable learning from sparse, heterogeneous features [9]. It can use item identifiers, problem metadata, and non-linear interactions among prior-performance variables without assuming a single latent state. In Uchko, XGBoost predicts near-term success while BKT answers a different question: how the system’s belief about a particular skill should change after new evidence.

The hybrid choice also supports graceful failure. If a problem identifier was not observed during fitting, the preprocessing pipeline ignores the unseen one-hot category while retaining history and metadata features. If a skill lacks sufficient evidence for a stable BKT fit, a global parameter set is used. These fallbacks are important for an application whose curriculum may evolve.

B. From Prediction to Selection

Purely maximizing predicted correctness would repeatedly offer easy items. Conversely, always choosing the lowest-

mastery skill may produce problems that are too difficult. Uchko uses a bounded heuristic inspired by the idea of maintaining productive challenge. Among candidates attached to the eight weakest available skills, the system evaluates up to five items per skill. For candidate item i , it minimizes

$$s_i = |\hat{p}_i - 0.65| + 0.10m_i, \quad (2)$$

where $\hat{p}_i$ is the XGBoost success probability and m_i is current BKT mastery. The first term strongly prefers a predicted success rate near 65%; the smaller second term breaks close decisions in favor of weaker skills. The twenty most recently attempted items are initially excluded, with a fallback if this removes every valid candidate. The 0.65 target is a transparent design parameter rather than a learned claim about optimal pedagogy.

III. DATA AND METHODOLOGY

A. Dataset and Responsible Use

FoundationalASSIST contains mathematics interactions aligned with Common Core standards and retains richer question and response information than conventional identifier-only knowledge-tracing benchmarks [5]. The raw release reports 3,395 problems and about 1.7 million interactions. Uchko’s training pipeline loads interaction identifiers, problem identifiers, hint use, answer-reveal indicators, binary scores, timestamps, and anonymized user identifiers. It joins these records to problem and skill metadata.

Rows with missing timestamps or non-binary scores are removed, interaction identifiers are deduplicated, and problems with ambiguous duplicate metadata records are excluded before a validated many-to-one join. The resulting dataset has 1,693,947 interactions from 5,000 students. The Django content importer produces 3,353 learning items, of which 2,483 have a supported, usable numeric or single-choice answer for adaptive practice. The curriculum view exposes 164 imported skills.

The interaction file is access-controlled data. It was processed only in the university environment, was not committed to the repository, and is not loaded into the deployed application database. Only trained artifacts, aggregate metrics, problem content, and skill metadata are used by the application. This separation reduces unnecessary exposure of historical student records.

B. Leakage-Aware Preprocessing

Interactions are ordered by student, timestamp, and interaction identifier. Every historical feature is computed strictly before the current target. The model uses five categorical variables—problem, problem set, problem type, answer type, and primary skill—and thirteen numerical variables. Numerical features include problem part, number of attached skills, prior user attempts and successes, Laplace-smoothed user accuracy, accumulated hints and answer reveals, previous correctness, previous hint use, capped time since the previous interaction, and analogous skill-specific counts and smoothed accuracy.

For example, the smoothed user accuracy before interaction t is

$$a_t = \frac{c_{<t} + 1}{n_{<t} + 2}, \quad (3)$$

where $c_{<t}$ and $n_{<t}$ include only earlier responses. This avoids undefined cold-start values and shrinks small histories toward 0.5.

Students, rather than rows, are shuffled with seed 42 and divided 70/15/15. The training partition contains 1,185,585 rows from 3,500 students; validation contains 253,894 rows from 750 students; and test contains 254,468 rows from 750 students. Student-disjoint evaluation is stricter than a random row split because the model cannot exploit the same learner identity across training and test data.

Categorical variables are one-hot encoded with rare levels grouped below a frequency of twenty. Numeric values are median-imputed and scaled without centering to preserve sparse matrices. The preprocessing object is fitted only on the training partition and serialized with the final feature schema.

C. Models and Evaluation

Three success-prediction references were constructed. A global baseline assigns the training-set success prevalence to every case. A history baseline uses the student’s smoothed prior accuracy. A linear logistic model, implemented with stochastic gradient descent, provides a validation-stage learned baseline. The selected XGBoost classifier uses a binary logistic objective, learning rate 0.05, maximum depth 8, row and column subsampling of 0.85, and up to 1,000 trees. Early stopping with patience 30 is performed only against validation log loss; the test partition is evaluated once after selection.

BKT parameters are fitted by bounded maximum likelihood. A global parameter vector is learned from all training sequences. A skill receives its own parameters only when the training data contain at least 1,000 interactions from at least 50 students; otherwise, the global vector is recorded as a fallback. This produces 95 skill-specific fits and 57 fallback entries. BKT predictions are generated sequentially, resetting state at student boundaries.

The comparative DKT model follows the recurrent formulation of Piech et al. [4]. Each interaction is encoded as a joint skill-response token and mapped to a 64-dimensional embedding. A single gated recurrent unit with 128 hidden units processes each student’s ordered history; a linear output layer predicts correctness for the next observed skill. Sequences are divided into chunks of at most 100 prediction steps with a one-event boundary overlap. The model contains 113,881 parameters and is optimized on CPU with Adam, learning rate 10^{-3} , batch size 128, dropout 0.15, weight decay 10^{-5} , and gradient clipping at 5. Training permits 15 epochs with patience three according to validation log loss. The best checkpoint occurs at epoch 15. DKT uses the same student partitions and target as the other predictors; fair model comparison excludes the first response of each test student, for which no prior sequence exists.

Evaluation reports threshold-independent ROC AUC together with accuracy and F1 at a 0.5 threshold. Because the adaptive policy consumes probabilities, log loss and Brier score are also reported. These proper scoring rules penalize poorly calibrated confidence rather than assessing ranking alone [10].

IV. SYSTEM IMPLEMENTATION

Figure 1 summarizes the implemented loop. Uchko is a server-rendered Django application backed by a relational database. The data model includes users with student or professor roles, courses, enrollments, knowledge components, learning items, learning sessions, attempts, and per-enrollment knowledge states. Database transactions and row locking protect mastery and attempt updates from inconsistent concurrent writes.

The implementation redesigns an earlier single-user Streamlit proof of concept as a persistent multi-user Django system, separating domain services, authorization, storage, and server-rendered presentation. Django authentication stores salted password hashes, CSRF middleware protects state-changing forms, and role- and ownership-aware checks separate student records from professor views. These measures make Uchko a production-oriented research prototype, although use with real learners would still require a dedicated security and privacy review.

At inference time, the application reconstructs the same features used during training from a student’s persisted history. The serialized preprocessor and XGBoost model run on CPU even though model fitting used GPU acceleration. Candidate items must have a supported answer and belong to an active course. After submission, answer checking supports tolerance-aware numeric responses and single-choice alternatives. DKT is retained as a reproducible comparison but is not deployed because it is less accurate than XGBoost and would add stateful sequence inference without an empirical benefit.

A critical implementation detail is the separation of *answer correctness* from *independent success*. If the final submitted answer is correct after a hint or after revealing the answer, the interface acknowledges correctness, but the attempt is not treated as independent mastery evidence. BKT is updated using

$$y^{\text{ind}} = y^{\text{answer}} \wedge (h = 0) \wedge \neg r, \quad (4)$$

where h is the hint count and r indicates that the answer was revealed. This choice aligns the online signal with the training target’s treatment of assistance and prevents a student from inflating mastery simply by requesting the answer. The interface explicitly explains why mastery may decrease after a correct but assisted response.

The student experience includes a role-aware home page, adaptive practice, progress summaries, curriculum exploration, and account information. Professors can inspect enrolled students and skill-level progress without accessing another course’s records. Imported question bodies and options can contain HTML and MathML, so they are sanitized through

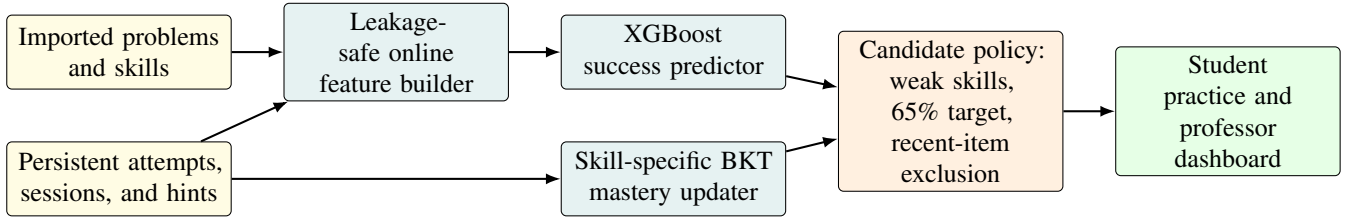


Fig. 1. Uchko’s hybrid adaptive loop. XGBoost predicts candidate success, BKT tracks skill mastery, and a deterministic policy converts both outputs into the next activity; the resulting response is persisted as new history.

TABLE I
MATCHED PERFORMANCE ON 750 HELD-OUT TEST STUDENTS

Model	AUC	Acc.	F1	Log loss	Brier
History	0.6932	0.6732	0.7602	0.6073	0.2097
DKT	0.7620	0.7167	0.7878	0.5542	0.1872
XGBoost	0.8125	0.7443	0.8060	0.5058	0.1692

explicit tag, attribute, CSS-property, URL-scheme, and image-host allowlists before rendering. Automated tests cover authorization, content sanitization, repeated-question scoring, and the adaptive workflow; continuous integration runs framework checks, migration checks, dependency checks, and the complete test suite.

V. RESULTS

Table I reports performance on the 253,718 test cases shared by the history, DKT, and XGBoost evaluations. The first response from each student is omitted because DKT requires a preceding interaction. The global validation baseline has an AUC of 0.5 by construction and is omitted for compactness. The linear model reaches 0.7868 AUC and 0.5393 log loss on validation, while validation results for XGBoost are 0.8087 and 0.5113. This advantage motivated selection of XGBoost before final testing.

DKT improves AUC by 0.0688 over matched history and reduces log loss by 8.7%, demonstrating that ordered skill-response sequences contain useful predictive structure. However, XGBoost exceeds DKT by 0.0505 AUC and reduces its log loss by a further 8.7%. XGBoost improves AUC by 0.1193 over matched history and lowers the Brier score by 19.3%. Its matched test AUC of 0.8125 is close to the original all-row test estimate of 0.8122, indicating that exclusion of each student’s first response does not materially change the conclusion.

BKT, evaluated sequentially on the full held-out test partition, obtains 0.6850 AUC, 0.6772 accuracy, 0.7671 F1, 0.6105 log loss, and 0.2106 Brier score. It does not outperform the history baseline as a next-response predictor. This result should not be hidden: BKT’s contribution is instead an interpretable state transition that can be updated after every response and inspected per skill. The deployed hybrid assigns prediction and state estimation to the models that best support each role.

The evaluation also illustrates why accuracy alone is insufficient. The training success prevalence is 0.6141, so an always-positive tendency can produce a superficially substantial F1

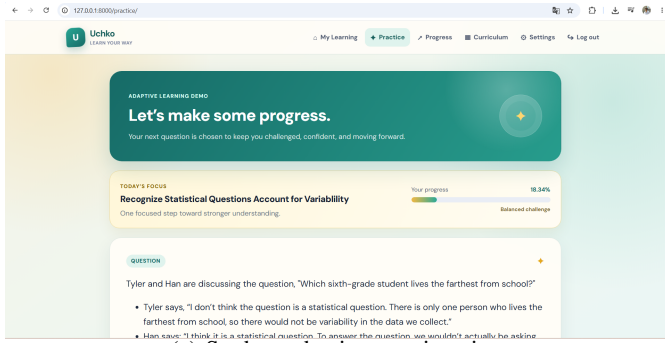
score. ROC AUC and proper probability losses more clearly distinguish the models. For the adaptive policy, probability quality is especially relevant because candidate ranking depends directly on distance from the 0.65 target.

VI. DISCUSSION

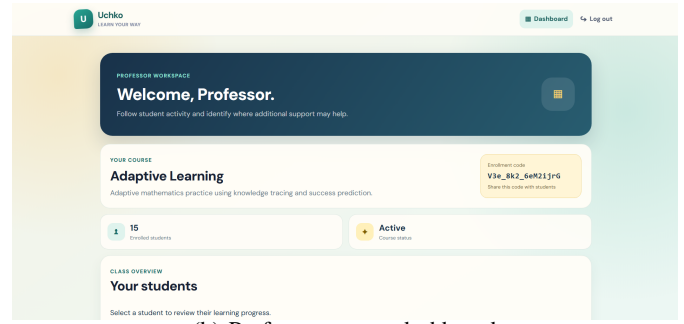
The experiment supports four conclusions. First, prior learner history and problem metadata together contain considerably more predictive information than aggregate history alone. XGBoost benefits from non-linear interactions among sparse item categories, temporal signals, hint histories, and skill-specific performance. Second, DKT’s gain over history confirms that a recurrent representation of ordered skill-response events is useful, but deep learning is not automatically the strongest choice: the feature-rich boosted model performs better on every reported metric. Third, student-disjoint partitions offer evidence that the models generalize to new learner accounts rather than memorizing students seen during training. Fourth, deployment requires design decisions that offline metrics do not resolve. The target probability, recent-item exclusion window, weakest-skill set size, and handling of assistance are transparent policy choices that require eventual user-study validation.

The study has limitations. FoundationalASSIST reflects one platform, mathematics curriculum, and historical logging policy. Offline next-response accuracy is not evidence of improved learning, retention, motivation, or fairness. The current split tests unseen students but does not simulate chronological dataset drift. XGBoost can express complex feature interactions, yet individual predictions are less pedagogically transparent than BKT states. The 0.65 success target is a reasonable design heuristic, not an experimentally optimized value. Finally, only numeric and single-choice items with recoverable answers are presently used in practice, reducing content coverage from 3,353 imported items to 2,483 usable items.

The DKT result is informative despite not becoming part of the deployed path. It was trained on the same student-disjoint split and evaluated against XGBoost on exactly the same next-response rows, eliminating a common source of misleading comparison. Its lower AUC and higher probabilistic losses do not show that recurrent knowledge tracing is generally ineffective; they show that this compact configuration and interaction representation do not outperform Uchko’s engineered prior-history and content features. Larger architectures,



(a) Student adaptive-practice view



(b) Professor course dashboard

Fig. 2. Role-aware Uchko interfaces. Students receive a focused next activity and progress signal, while professors see enrollment-level activity and can open individual progress profiles. The displayed population is synthetic demonstration data.

longer optimization, and attentive knowledge tracing remain possible extensions, but would require new validation rather than deployment by assumption.

VII. CONCLUSION

Uchko demonstrates an end-to-end path from protected educational interaction data to adaptive web-based practice. On unseen students, XGBoost substantially outperforms aggregate history and a compact recurrent DKT model for next-response prediction, while BKT provides a simple and inspectable per-skill state. The negative DKT comparison supports retaining the more accurate and simpler deployed predictor instead of selecting a model by architectural novelty. A deterministic policy combines prediction and mastery to focus on weak skills at a controlled level of challenge. Persistent data models, role-based authorization, safe content rendering, and explicit treatment of assisted answers turn the experiment into a functioning platform rather than an isolated notebook. The remaining research question is educational impact: future work should test whether the policy improves learning and whether its recommendations remain fair, understandable, and useful in authentic classrooms.

REFERENCES

- [1] T. Kabudi, I. O. Pappas, and D. H. Olsen, “AI-enabled adaptive learning systems: A systematic mapping of the literature,” *Computers and Education: Artificial Intelligence*, vol. 2, p. 100017, 2021.
- [2] L. Y. Tan, S. Hu, D. J. Yeo, and K. H. Cheong, “Artificial intelligence-enabled adaptive learning platforms: A review,” *Computers and Education: Artificial Intelligence*, vol. 7, p. 100429, 2025.
- [3] A. T. Corbett and J. R. Anderson, “Knowledge tracing: Modeling the acquisition of procedural knowledge,” *User Modeling and User-Adapted Interaction*, vol. 4, pp. 253–278, 1994.
- [4] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. J. Guibas, and J. Sohl-Dickstein, “Deep knowledge tracing,” in *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [5] E. Worden, C. Heffernan, N. Heffernan, and S. Sonkar, “Foundation-aLASSIST: An educational dataset for foundational knowledge tracing and pedagogical grounding of LLMs,” *arXiv preprint arXiv:2602.00070*, 2026.
- [6] J. Roschelle, M. Feng, R. F. Murphy, and C. A. Mason, “Online mathematics homework increases student achievement,” *AERA Open*, vol. 2, no. 4, 2016.
- [7] A. Ghosh, N. Heffernan, and A. S. Lan, “Context-aware attentive knowledge tracing,” in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2020, pp. 2330–2339.
- [8] G. Abdelrahman, Q. Wang, and B. P. Nunes, “Knowledge tracing: A survey,” *ACM Computing Surveys*, vol. 55, no. 11, 2023.
- [9] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [10] G. W. Brier, “Verification of forecasts expressed in terms of probability,” *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.